Java

Язык, платформа и модель исполнения



Backend-программирование

Что разберём сегодня

- Откуда появилась Java и какие задачи на ней решают
- Чем отличаются Java, JVM, JRE и JDK
- Что происходит между исходным кодом и выполнением программы
- Как JVM управляет памятью и зачем нужен GC

Результат: вы можете объяснить, что нужно для сборки и запуска Java-приложения.

Определение Java

«Java — строго типизированный объектно-ориентированный язык программирования общего назначения, разработанный компанией Sun Microsystems (в последующем приобретённой компанией Oracle)».

Википедия, статья «Java» · цитата первого предложения

За длинным определением стоят конкретные свойства. Разберём их по отдельности.

Что означает это определение

Термин	Смысл
Общего назначения	Можно писать разные виды программ
Объектно-ориентированный	Классы описывают данные и поведение объектов
Статически типизированный	Типы выражений и допустимые операции проверяет компилятор
Управляемая среда	Среда исполнения берёт на себя часть работы с памятью

Компилятор способен найти часть ошибок ещё до запуска.

Краткая история: Sun Microsystems

Период	Событие
Начало 1990-х	Джеймс Гослинг и команда Sun создают язык Oak
1995	Язык получает имя Java и выходит публично
1996	Выходит JDK 1.0
Изначальная идея	Одна программа для разных устройств

Переносимость появилась в Java как одна из исходных целей.

От браузерных апплетов к серверам

- В 1990-е Java-апплеты добавляли интерактивность веб-страницам
- Серверные приложения стали одним из основных направлений Java
- 2006–2007: Sun открывает основные исходники, развивается OpenJDK
- 2010: Oracle приобретает Sun Microsystems

Современная серверная Java не требует Java-плагина в браузере.

Кто развивает Java

Участник	Роль
JCP и спецификации	Описывают требования к языку и платформе
OpenJDK	Совместная разработка открытой реализации
Oracle и сообщество	Развитие языка, библиотек и JVM
Поставщики JDK	Сборки, обновления и поддержка для пользователей

Одна платформа может иметь несколько совместимых реализаций и сборок.

Где используется Java

Область	Примеры
Серверные приложения	API, банковские сервисы, учётные системы
Хранение и обработка данных	Apache Cassandra, Apache Kafka, Elasticsearch
Инструменты разработки	Jenkins, IntelliJ IDEA, Gradle
Игры	Minecraft: Java Edition

На Java пишут и бизнес-приложения, и инфраструктуру для других разработчиков.

Инфраструктура: три разные задачи



Cassandra

Распределённая
NoSQL-база данных



Kafka

Платформа
потоков событий



Elasticsearch

Поисковая система
и аналитика

Все три продукта используют JVM, но решают разные задачи.

Java в знакомых инструментах



Jenkins
Сборка и автоматизация



IntelliJ IDEA
Среда разработки



Gradle
Система сборки

Ещё один знакомый пример: Minecraft: Java Edition.

Почему Java выбирают для backend

- Совместимость помогает развивать системы годами
- Статическая типизация упрощает проверку и изменение большого кода
- Библиотеки закрывают типовые серверные задачи
- JIT, многопоточность и GC подходят для длительно работающих сервисов
- Отладчики и профилировщики помогают находить проблемы

Преимущество Java складывается из языка, среды исполнения и экосистемы.

Статическая типизация: ранняя проверка

```
int age = 20;       // допустимо
age = "двадцать";     // ошибка типа
```

Компилятор знает тип переменной и проверяет допустимость значения.

Логические ошибки и неверные данные всё равно требуют проверок и тестов.

Совместимость и стабильность

- Старый байткод часто работает на более новой JVM
- Более старая JVM может не понять новый формат class-файла
- Переход между версиями может потребовать изменений
- Стабильность API снижает цену сопровождения большого проекта

Совместимость имеет направление и ограничения.

Цена преимуществ Java

- Среде исполнения нужна память сверх памяти объектов приложения
- Запуск JVM и прогрев JIT занимают время
- Сборка мусора расходует CPU и может приостанавливать приложение
- Библиотеки, версии и сборка тоже требуют внимания

Выбор технологии зависит от задачи: универсального победителя нет.

Четыре значения слова Java

Название	Что имеется в виду
Язык Java	Правила записи исходного кода
Платформа Java	Среда исполнения, библиотеки и спецификации
JDK	Устанавливаемый комплект для разработки
Команда java	Программа запуска Java-приложений

Контекст важен: «установить Java» обычно означает выбрать конкретный дистрибутив.

JVM, JRE и JDK

Термин	Расшифровка	Назначение
JVM	Java Virtual Machine	Исполнение class-файлов
JRE	Java Runtime Environment	Среда запуска: JVM + библиотеки
JDK	Java Development Kit	Разработка: runtime + инструменты

Для разработки устанавливаем JDK. Для запуска нужна совместимая среда исполнения.

JVM: виртуальная машина Java

- Читает и проверяет class-файлы
- Выполняет инструкции и управляет объектами
- Взаимодействует с операционной системой через свою реализацию
- Имеет разные реализации: например, HotSpot и Eclipse OpenJ9

JVM работает внутри процесса операционной системы.

Что входит в JDK

Инструмент	Задача
javac	Компилирует исходный код
java	Запускает приложение
javap	Показывает устройство class-файла
jar	Создаёт и читает Java-архивы
jshell	Выполняет фрагменты Java интерактивно

IDE использует эти возможности и добавляет удобный интерфейс.

Почему JRE не всегда скачивают отдельно

- JRE остаётся полезным термином для среды запуска
- Начиная с JDK 11 Oracle не выпускает отдельный JRE
- Некоторые другие поставщики продолжают распространять JRE
- Приложение может поставляться со своей средой исполнения

Современная поставка может содержать JDK или специально собранный runtime.

Кто поставляет JDK

Поставщик / проект	Пример дистрибутива
Oracle	Oracle JDK, Oracle OpenJDK builds
Eclipse Adoptium	Eclipse Temurin
Amazon	Amazon Corretto
Azul	Azul Zulu
Microsoft	Microsoft Build of OpenJDK
IBM	IBM Semeru Runtimes

Сравнивают версию, ОС, архитектуру, условия использования и поддержку.

Версия, вендор и архитектура

Temurin JDK 21 macOS AArch64

Часть имени	На какой вопрос отвечает
Temurin	Какой дистрибутив?
JDK 21	Какая версия платформы?
macOS	Для какой операционной системы?
AArch64	Для какой архитектуры процессора?

Установленный JDK и JDK, выбранный в IDE, могут оказаться разными.

Релизы и LTS

- Новые функциональные релизы Java выходят каждые полгода
- LTS означает длительную поддержку у конкретного поставщика
- Примеры LTS-версий: 17, 21 и 25
- Сроки обновлений и условия поддержки зависят от дистрибутива

Версию проекта выбирают осознанно и фиксируют в настройках сборки.

Java как платформа для других языков



Kotlin



Scala



Clojure



Groovy

Разные языки используют общую JVM и могут обращаться к Java-библиотекам.

Изучение платформы пригодится даже при смене JVM-языка.

Библиотеки и фреймворки

Слой	Примеры задач
Стандартная библиотека	Коллекции, файлы, даты, сеть
Внешняя библиотека	JSON, драйвер базы данных, тестирование
Фреймворк	Каркас приложения: Spring, Quarkus
Система сборки	Зависимости, компиляция, тесты, упаковка

JDK даёт основу. Остальные инструменты подключают по потребности.

Путь Java-программы



Компиляция исходника и запуск программы происходят на разных этапах.

Hello World

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Hello.java: main начинает выполнение, println выводит строку.

Компиляция и запуск

```
javac Hello.java

java Hello
Hello, world!
```

javac
создаёт Hello.class

java
запускает класс Hello

Изменили исходник? Для этого способа запуска нужно снова выполнить javac.

Что находится в class-файле

- Версия формата
- Описание класса, полей и методов
- Инструкции байткода
- Таблица констант и другая служебная информация

```
javap -c Hello
```

Байткод можно изучать, не открывая бинарный файл в текстовом редакторе.

Пример: Java и байткод

Исходный код

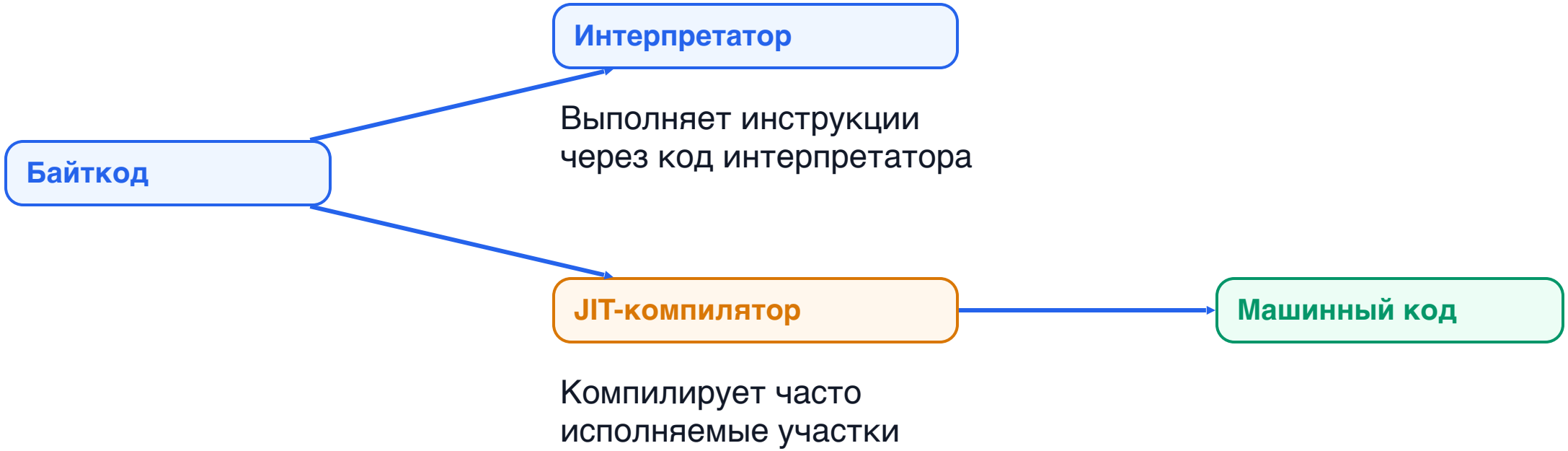
```
static int sum(int a, int b) {
    return a + b;
}
```

Фрагмент javap -c

```
0: iload_0
1: iload_1
2: iadd
3: ireturn
```

Байткод состоит из инструкций JVM, а не из инструкций конкретного процессора.

Как JVM выполняет байткод



JIT = Just-In-Time: компиляция во время работы приложения.

JIT и прогрев

- JVM собирает информацию о реальном выполнении
- JIT оптимизирует часто используемые методы и циклы
- Первые запросы могут вести себя иначе, чем запросы после прогрева
- Оптимизация требует времени и памяти

Одного запуска без прогрева недостаточно для оценки производительности сервиса.

Как обычно компилируют программу на C



Исходный текст

Создаёт объектный код для целевой платформы

Связывает объектные файлы и библиотеки

Нативный исполняемый файл

```
cc hello.c -o hello
./hello
```

В типичной сборке C машинный код получают до запуска программы.

Типичная сборка: C и Java

Вопрос	C: обычная нативная сборка	Java: javac + JVM
Результат сборки	Бинарник для целевой ОС/CPU	Class-файлы или JAR
Что нужно при запуске	ОС и нужные библиотеки	Совместимый runtime
Где машинный код	В бинарнике до запуска	В JVM и в коде, созданном JIT
Перенос на другую ОС	Обычно новая сборка	Часто тот же байткод

В обоих случаях процессор в итоге выполняет машинные инструкции.

Write once, run anywhere: условия

- Для целевой ОС и процессора есть подходящая JVM
- JVM понимает версию class-файла
- Все необходимые библиотеки доступны
- Код не зависит от специфичных путей, команд или нативных библиотек

Переносимость помогает, но не отменяет проверку на целевой системе.

Задача: что отправить на сервер?

На ноутбуке приложение собрано для Java 21.
Сервер работает на Linux, но на нём установлена Java 17.

- Нужен ли серверу исходный код?
- Достаточно ли скопировать class-файлы или JAR?
- Нужно ли пересобирать приложение только из-за другой ОС?
- Что делать с несовпадением версий?

Разбор: запуск на другом компьютере

- Исходник обычно не нужен для запуска готового артефакта
- Нужны артефакт, зависимости, конфигурация и подходящий runtime
- Смена ОС сама по себе часто не требует перекомпиляции Java-кода
- Class-файл для Java 21 не запустится на JVM 17

Решение: подходящая JVM или сборка под 17, если код и библиотеки это допускают.

JAR: упаковка приложения

- Архив классов и ресурсов
- Может быть библиотекой или запускаемым приложением
- Может содержать manifest с указанием точки входа
- Зависимости не обязательно включены внутрь

```
java -jar app.jar
```

JAR удобен для передачи приложения, но наличие файла ещё не гарантирует запуск.

Ошибка сборки и ошибка выполнения

- Ошибка компиляции: исходник нарушает правила языка или типов
- Ошибка запуска: не найден класс, зависимость или подходящая JVM
- Ошибка во время работы: неверные данные, сеть или логика программы

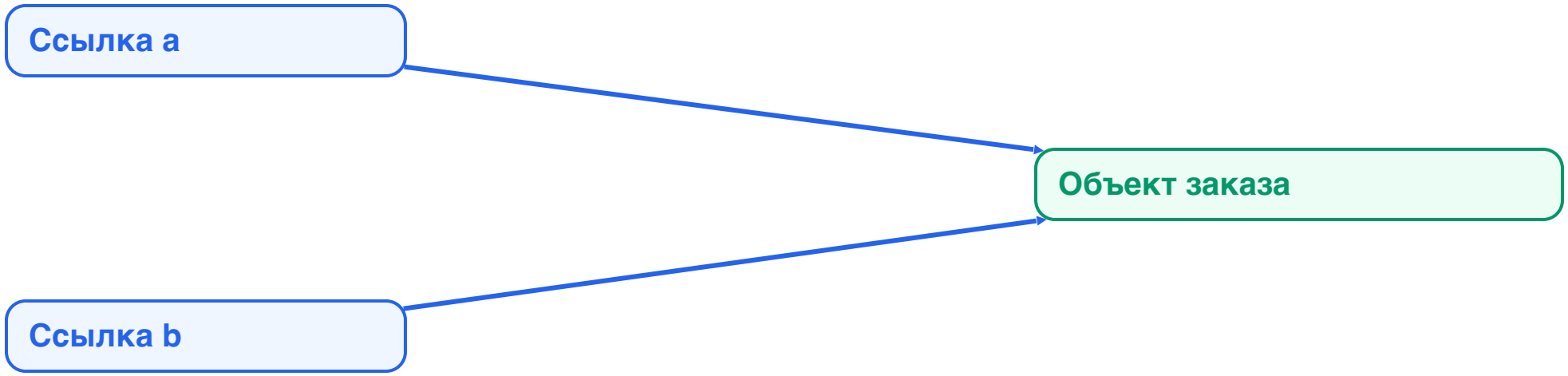
Первый вопрос при разборе ошибки: на каком этапе она возникла?

Память работающего приложения

Область	Что хранит
Heap / куча	Объекты и массивы приложения
Стеки потоков	Кадры вызовов, локальные значения и ссылки
Метаданные классов	Информацию о загруженных классах
Code cache и другие области	Машинный код JIT и служебные данные

Память процесса JVM не ограничивается кучей объектов.

Объект и ссылка на объект



Две ссылки

Один объект

Удаление одной ссылки не делает объект недоступным, если осталась другая.

Garbage Collection: сборка мусора

- Программа создаёт объекты по мере работы
- Часть объектов со временем становится недостижимой
- GC находит такие объекты и освобождает занятую ими память
- Разработчик обычно не вызывает free для Java-объектов

GC освобождает память автоматически, но момент сборки не фиксирован.

Достижимость: какие объекты живы?



Есть путь от корней

Нет пути от корней



Взаимные ссылки C и D не делают их достижимыми от корней.

Что происходит при сборке

- Сборщик определяет, какие объекты доступны программе
- Освобождает или повторно использует память недостижимых объектов
- Некоторые сборщики перемещают живые объекты
- Часть работы может идти параллельно приложению, часть требует пауз

У GC несколько алгоритмов. Единого сценария для всех JVM нет.

Что GC не решает

- Не удаляет объекты, которые приложение продолжает удерживать
- Не исправляет неограниченно растущий кэш
- Не заменяет своевременное закрытие файлов и соединений
- Не гарантирует отсутствие OutOfMemoryError

Память объектов и внешние ресурсы требуют разных правил управления.

Мини-задача: почему растёт память?

Сервис после каждого запроса добавляет его данные в общий список. Список никогда не очищают.

- Почему GC не освобождает эти данные?
- Поможет ли просто чаще запускать сборку?
- Как изменить поведение программы?

Разбор: GC сохраняет доступные данные

- Общий список остаётся доступным приложению
- Элементы списка достижимы через него
- GC не знает, что бизнес-логике они больше не нужны
- Нужны ограничение размера, удаление или другая модель хранения

Частая сборка не освобождает живые объекты.

GC и серверная нагрузка

Критерий	Вопрос
Задержка	Как долго отдельный запрос может ждать?
Пропускная способность	Сколько полезной работы выполняется за секунду?
Потребление памяти	Какой объём памяти допустим?

Настройки GC оценивают по измерениям приложения.

Что нужно унести с занятия

- Java обозначает и язык, и платформу
- JDK нужен для разработки, runtime для запуска, JVM для исполнения
- Обычный javac создаёт байткод, а JIT может создавать машинный код
- Переносимость зависит от runtime, версии и зависимостей
- GC работает с достижимостью и не исправляет удержание данных

Следующее занятие: переменные, типы, выражения, условия, циклы и методы.

Дополнительно: сборщики мусора

Сборщик	Основной ориентир
Serial GC	Простота, небольшие объёмы работы
Parallel GC	Пропускная способность
G1 GC	Баланс и целевые ограничения пауз
ZGC	Низкие задержки сборки

Для первого проекта важнее корректное поведение программы, чем ручной тюнинг GC.

Дополнительно: Java SE и Jakarta EE

- Java SE задаёт базовую платформу и стандартные API
- Jakarta EE развивает спецификации для корпоративных приложений
- Историческое название Jakarta EE: Java EE
- Spring является отдельной экосистемой и использует Java-платформу

JDK, Jakarta EE и Spring решают задачи разных уровней.

Дополнительно: Android и нативная сборка

- Android использует ART и формат DEX
- Синтаксис Java на Android не означает обычный запуск class-файлов
- GraalVM Native Image заранее собирает Java-приложение в нативный файл
- Нативная сборка меняет поставку и накладывает свои ограничения

Байткод + JVM + JIT — основной путь этой лекции, но возможны и другие.

Источники и дальнейшее чтение

Источник	Адрес
Dev.java	dev.java/learn/
Спецификация JVM	docs.oracle.com/javase/specs/
JDK Tools и руководство по GC	docs.oracle.com/en/java/javase/
OpenJDK и дистрибутивы	openjdk.org · adoptium.net
Википедия: цитата определения	ru.wikipedia.org/wiki/Java

Вопросы?